FIAP · ENGENHARIA DE SOFTWARE · 4ESR-2026

CP05 — O Pipeline DevSecOps Completo

DevSecOps and Cloud Computing

Pipeline de segurança de referência para a Arcanjosa (empresa fictícia): três barreiras automáticas no GitHub Actions — **Gitleaks** (segredos), **Semgrep** (código vulnerável) e **Trivy** (imagem Docker com CVE crítico) — e as provas de sabotagem que mostram cada barreira funcionando.

Aluno	**Eduardo Akira Murata**
RM	**98713**
Repositório	github.com/akiramrt/cp05-devsecops-pipeline
Workflow	.github/workflows/security.yml
Data	22 de outubro de 2026

1. Visão geral

Uma chave de API vazou num commit público da Arcanjosa, e a diretoria tornou obrigatório um pipeline de segurança em todos os repositórios. Este trabalho entrega o **pipeline de referência**: um workflow do GitHub Actions que roda em todo `push` e todo `pull_request` e barra três tipos de problema antes que eles cheguem ao `main`: **segredos** no histórico do Git (Gitleaks), **código vulnerável** (Semgrep) e **imagem Docker com CVE crítico** (Trivy). Para provar que o pipeline barra de verdade, cada barreira foi sabotada de propósito num branch separado, com um pull request que ficou bloqueado.

push / PR	**gitleaks**	**semgrep**	**docker build + trivy**	**main protegido**
gatilho do workflow	segredos nos commits	SAST + regra no-eval	CVE CRITICAL com correção	3 checks obrigatórios

Mapa da entrega

O que o CP05 pede	Onde está
1. Repositório com Dockerfile + site	akiramrt/cp05-devsecops-pipeline: `Dockerfile`, `site/`, `nginx/default.conf` (seção 2)
2. Workflow `security.yml` com 3 jobs	`.github/workflows/security.yml`: jobs `gitleaks`, `semgrep` e `trivy` (seção 3 e Anexo A)
3. Prova 1: segredo falso BARRADO pelo Gitleaks	PR #1, execuções #2 e #3: seção 5, Figuras 3 e 4
4. Prova 2: `eval` APONTADO pelo Semgrep	PR #2, execuções #4 e #6: seção 6, Figuras 5 e 6
Trivy: imagem limpa PASSANDO	Execução #1 no `main`: Figura 1, log da seção 4 e seção 7
5. PDF com prints das Actions + parágrafo por barreira	Este documento. Cada barreira tem um quadro "O que detectou / Por que é perigoso" (seções 5, 6 e 7), e cada print traz usuário, repositório e horário na legenda
Bônus: imagem vulnerável BARRADA pelo Trivy	PR #3, execuções #5 e #7: seção 7, Figuras 7 e 8
Merge bloqueado pelos checks	Proteção do `main`, inclusive para o administrador (seção 8)

Sobre os prints: foram tirados da visão pública do GitHub, sem login, para que qualquer pessoa possa conferir as mesmas páginas pelos links. Por isso aparecem "Sign in" e horários relativos ("2 minutes ago"). O horário absoluto de cada execução e de cada captura está na legenda, e os logs completos (que o GitHub só mostra logado) foram baixados pela API e aparecem como trechos com horário.

2. O repositório: site + Dockerfile

Arquivo	O que faz
`site/`	Site estático da Arcanjosa (HTML, CSS e JS). O JS só usa `textContent`: nenhum `eval` nem `innerHTML`.
`nginx/default.conf`	nginx na porta 8080 com CSP sem `'unsafe-eval'`, `X-Frame-Options`, `nosniff`, COOP/CORP, `/healthz` e bloqueio de arquivos ocultos (`.git`, `.env`).
`Dockerfile`	Base `nginx/nginx-unprivileged:1.31.8-alpine-slim` fixada por digest, roda como usuário 101 (não-root), arquivos do site com dono root (somente leitura para o nginx) e `HEALTHCHECK`.
`.dockerignore`	Lista de permissão: só `site/` e a config do nginx entram na imagem; arquivos ocultos ficam de fora.
`.semgrep/no-eval.yml`	Regra própria do Semgrep: proíbe `eval()`, `new Function()` e timers com string literal, como `setTimeout("…")` (CWE-95).
`.github/scripts/sarif_report.py`	Converte o relatório SARIF de cada ferramenta em anotações (arquivo e linha) na aba Actions e no diff do PR.

```
# Imagem base: nginx oficial "unprivileged" (roda como usuário não-root na porta 8080),
# variante alpine-slim (menos pacotes = menos superfície de ataque e menos CVEs).
# Fixada por versão + digest: o build é reproduzível e uma tag republicada não muda a imagem.
FROM nginx/nginx-unprivileged:1.31.8-alpine-
slim@sha256:a2ae214d0266826427b804b0513dce20c9c93b862f1593203f9e6b50be9c2689

LABEL org.opencontainers.image.title="arcanjosa-site" \
      org.opencontainers.image.description="Site da Arcanjosa - CP05 DevSecOps (FIAP)" \
      org.opencontainers.image.authors="Eduardo Akira Murata - RM 98713" \
      org.opencontainers.image.source="https://github.com/akiramrt/cp05-devsecops-pipeline"

# Config do nginx com headers de segurança e o endpoint /healthz
COPY nginx/default.conf /etc/nginx/conf.d/default.conf

# Conteúdo estático. Os arquivos ficam com dono root, então o processo do nginx
# (usuário 101) consegue ler, mas não consegue alterar o site.
COPY site/ /usr/share/nginx/html/

USER 101

EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget -q --spider http://127.0.0.1:8080/healthz || exit 1
```

3. O workflow `security.yml`

O workflow segue o esqueleto da aula (`name: Security Pipeline`, `on: [push, pull_request]`) e tem três jobs independentes que rodam em paralelo. Cada job tem o nome que a proteção do branch exige (`gitleaks`, `semgrep`, `trivy`). O YAML completo está no Anexo A.

Job	Como funciona	Quando falha
gitleaks	Checkout com histórico completo; `gitleaks/gitleaks-action` v3.0.0 com o Gitleaks 8.30.1. Varre só os commits do push ou do PR e comenta	Qualquer segredo encontrado (código de

	no PR, na linha do segredo.	saída 2).
semgrep	Container `semgrep/semgrep:1.181.1` (fixado por digest). `semgrep scan --config p/default --config .semgrep/ --error`: as 1.074 regras do pacote da comunidade mais a regra própria. Dessas, 507 se aplicam às linguagens do repositório e rodam no scan. O SARIF vai para a aba Security > Code scanning.	Qualquer achado (`--error` sai com código 1).
trivy	`docker build` da imagem do site; `aquasecurity/trivy-action` v0.36.0 com o Trivy 0.76.1 (`--severity CRITICAL --ignore-unfixed --exit-code 1`). Depois, um smoke test sobe o container (read-only, sem capabilities) e testa `nginx -t`, `/healthz` e os headers.	CVE CRITICAL que já tem correção, ou smoke test quebrado.

Decisões de segurança do próprio pipeline

- **Actions fixadas por SHA completo.** Em março de 2026, 76 das 77 tags do `aquasecurity/trivy-action` foram reescritas por atacantes para apontar para um código que roubava segredos dos runners (advisory GHSA-69fq-xp46-6x23 / CVE-2026-33634). Quem referenciava uma tag e rodou o workflow nessa janela (19 e 20/03/2026) executou o malware. Fixar o SHA protegeu quem usava um commit recente da action, mas o advisory lista como afetados até SHAs anteriores a 09/04/2025: esses commits ainda baixavam o `aquasecurity/setup-trivy` por tag, e as tags dele também foram reescritas. Ou seja, o SHA trava o código da action, não o que ela baixa por tag. Por isso todo `uses:` aqui é um SHA de 40 caracteres (com a versão em comentário), conferi que a v0.36.0 chama o setup-trivy por SHA, e a imagem do Semgrep é fixada por digest.
- **Ferramentas em versões sem CVE conhecido.** Sem a variável `GITLEAKS_VERSION`, a action usaria o Gitleaks 8.24.3, afetado pelo CVE-2026-63728; aqui é o 8.30.1. A versão do Trivy (0.76.1) também é explícita, nunca `latest`.
- **Menor privilégio.** O token do workflow só lê o repositório. O job gitleaks ganha `pull-requests: write` (para comentar no PR); semgrep e trivy ganham `security-events: write` (para enviar SARIF); `persist-credentials: false` não deixa o token salvo no checkout.

Bug encontrado e corrigido: o "verde falso" do Gitleaks

Testando o pipeline localmente, descobri que a gitleaks-action monta o intervalo `<primeiro commit do push>^..<último>`. Se um push com dois ou mais commits começa no commit raiz do repositório, `raiz^` não existe: o git dá erro, o Gitleaks varre **0 commits**, imprime "no leaks found" e o job passa **verde sem ter verificado nada**. Um pipeline de segurança tem que falhar fechado (*fail-closed*: na dúvida, reprova), então adicionei o passo "Guarda": ele lê o evento do push e reprova o job nesse caso, antes do Gitleaks rodar. Com um commit só, a action usa `--log-opts=-1` e varre normalmente, como na execução #1.

4. Execução de referência: `main` verde

O primeiro push do pipeline no `main` (execução #1, iniciada em 22/10/2026 18:01:51) passou nos três jobs. Isso mostra que o pipeline não é "sempre vermelho": o código não tem achados e a imagem não tem CVE CRITICAL com correção. Só a sabotagem faz cada barreira falhar.

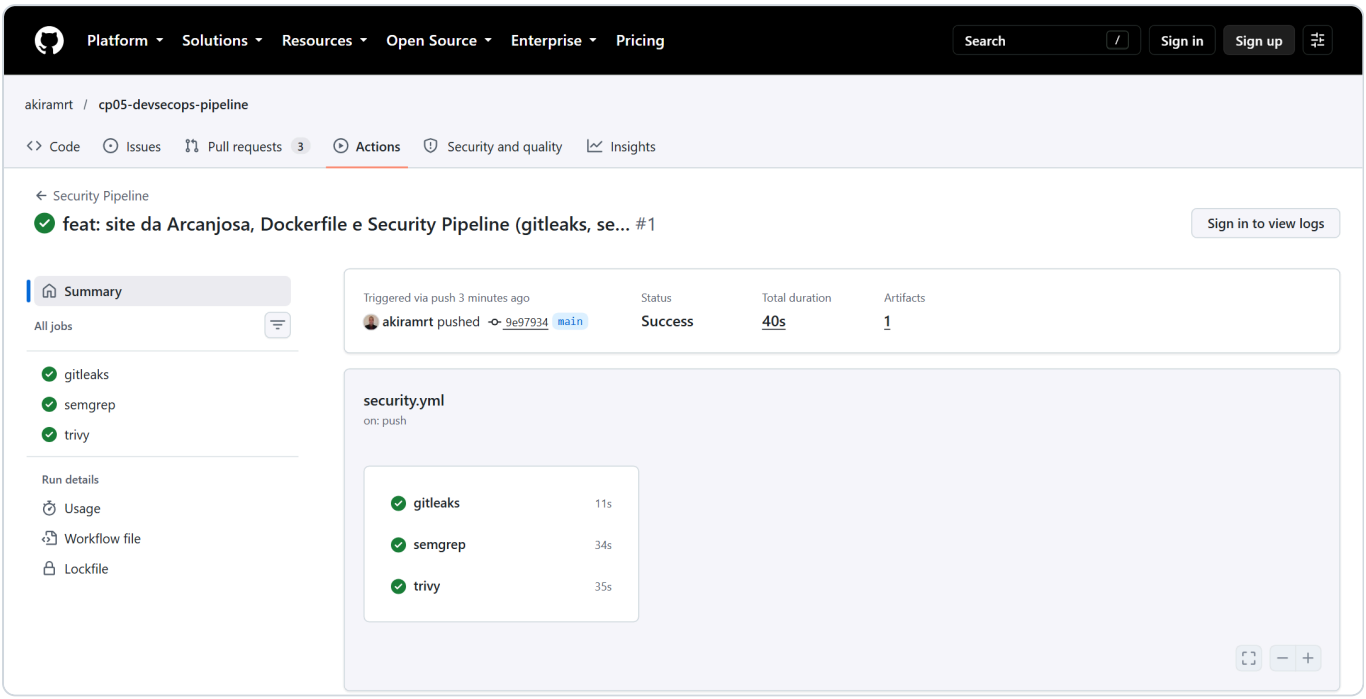


Figura 1: execução #1 de akiramrt/cp05-devsecops-pipeline no `main`, commit `9e97934`, push de akiramrt, iniciada em 22/10/2026 18:01:51 (Brasília); print às 18:05:07. gitleaks, semgrep e trivy aprovados · github.com/akiramrt/cp05-devsecops-pipeline/actions/runs/39853873641

Log do job `gitleaks / semgrep / trivy` da execução #1, baixado pela API do GitHub (horários em UTC; Brasília = UTC−3).

```
21:01:57  Commits neste push: 1 (primeiro: 9e97934821d1a871e3ed8f478cb2277c04555761)
21:02:00  gitleaks cmd: gitleaks detect --redact -v --exit-code=2 --report-format=sarif --report-path=results.sarif --log-level=debug --lo
21:02:01  9:02PM INF 1 commits scanned.
21:02:01  9:02PM INF no leaks found
```

```
21:02:15  Ran 507 rules on 13 files: 0 findings.
21:02:16  Semgrep: 0 achado(s) em semgrep.sarif
```

```
21:02:15  Report Summary
21:02:15
21:02:15
21:02:15  ┌──────────────────────────────────────────────────────────────────────────┬────────┬─────────────────┐
21:02:15  │                                  Target                                  │  Type  │ Vulnerabilities │
21:02:15  ├──────────────────────────────────────────────────────────────────────────┼────────┼─────────────────┤
21:02:15  │ arcanjosa-site:9e97934821d1a871e3ed8f478cb2277c04555761 (alpine 3.24.2)  │ alpine │        0        │
21:02:15  └──────────────────────────────────────────────────────────────────────────┴────────┴─────────────────┘
21:02:15
21:02:15  nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
21:02:15  nginx: configuration file /etc/nginx/nginx.conf test is successful
21:02:17  ok
21:02:17  ok
21:02:17  HTTP/1.1 200 OK
21:02:17  Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self'; object-src 'none'; base-uri 'r
21:02:17  X-Content-Type-Options: nosniff
21:02:17  X-Frame-Options: DENY
```

4.1 Todas as execuções

Cada sabotagem foi enviada num branch próprio, criado a partir do `main`, e virou um pull request. O workflow roda duas vezes para cada uma: no push do branch e no PR. Em todas, **só a barreira sabotada falhou**. As outras duas continuaram verdes, o que mostra que cada barreira é específica.

Execução	Prova	Branch · evento · commit	Início (Brasília)	gitleaks	semgrep	trivy
#1	referência	`main` push · `9e97934`	22/10/2026 18:01:51	PASSOU	PASSOU	PASSOU
#2	Prova 1	`sabotagem/01-gitleaks-segredo-falso` push · `9e64b51`	22/10/2026 18:03:34	FALHOU	PASSOU	PASSOU
#3	Prova 1	`sabotagem/01-gitleaks-segredo-falso` pull_request (PR #1) · `9e64b51`	22/10/2026 18:03:54	FALHOU	PASSOU	PASSOU
#4	Prova 2	`sabotagem/02-semgrep-eval` push · `31edbc4`	22/10/2026 18:04:27	PASSOU	FALHOU	PASSOU
#6	Prova 2	`sabotagem/02-semgrep-eval` pull_request (PR #2) · `31edbc4`	22/10/2026 18:04:51	PASSOU	FALHOU	PASSOU
#5	Bônus	`sabotagem/03-trivy-imagem-vulneravel` push · `780b8ca`	22/10/2026 18:04:42	PASSOU	PASSOU	FALHOU
#7	Bônus	`sabotagem/03-trivy-imagem-vulneravel` pull_request (PR #3) · `780b8ca`	22/10/2026 18:04:56	PASSOU	PASSOU	FALHOU

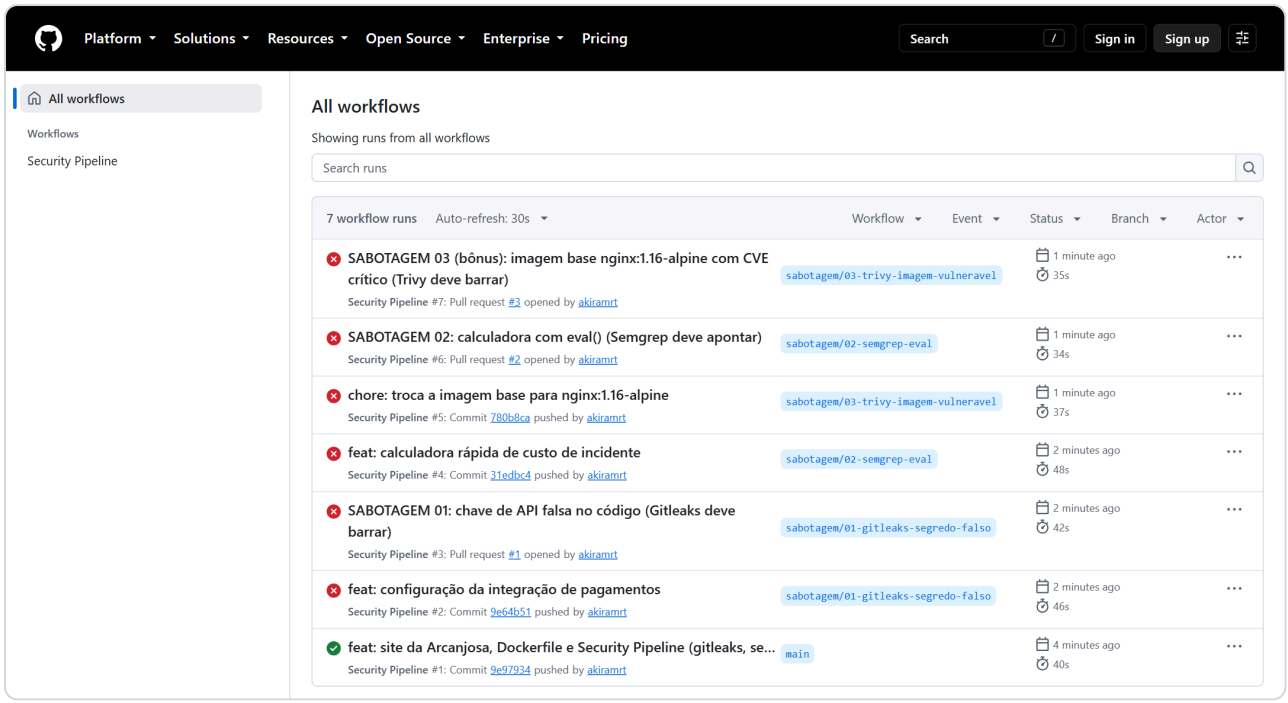


Figura 2: aba Actions de akiramrt/cp05-devsecops-pipeline, print de 22/10/2026 18:06:31 (Brasília): as 7 execuções das provas, de 18:01:51 (#1) a 18:04:56 (#7), todas de akiramrt · github.com/akiramrt/cp05-devsecops-pipeline/actions

5. Barreira 1: Gitleaks (segredos) SEGREDO BARRADO

Sabotagem. No branch `sabotagem/01-gitleaks-segredo-falso`, o commit `9e64b51` adicionou `site/js/config.js` com a constante `PAYMENT_API_KEY` e uma chave de API **falsa** de 31 caracteres escrita direto no código. É o mesmo erro do incidente da Arcanjosa.

O que o Gitleaks detectou e por que isso é perigoso

O que detectou. O Gitleaks não olha só os arquivos atuais: ele lê o histórico do Git (`git log -p`) dos commits enviados no push ou no PR e testa cada linha adicionada contra suas regras de credenciais, que combinam expressão regular, entropia de Shannon e listas de exceção. A linha 7 de `site/js/config.js` casou com a regra `generic-api-key`: um nome que contém "API_KEY", uma atribuição e um valor de 31 caracteres com entropia 4,95. Essa entropia está acima do limite de 3,5 da regra, ou seja, a string é aleatória demais para ser texto comum. O Gitleaks saiu com código 2, o job falhou, a action comentou no PR e, como `gitleaks` é um check obrigatório do `main`, o merge ficou bloqueado. Repare que o log mostra `REDACTED`: com `--redact`, o Gitleaks não copia a chave para logs, relatórios e artefatos. Ela ainda aparece no diff da Figura 4 porque está no próprio commit, e é exatamente isso que torna o vazamento grave (aqui a chave é falsa).

Por que é perigoso. Repositórios públicos são varridos por robôs o tempo todo, e uma chave exposta permite usar o serviço em nome da empresa (custos, acesso a dados de clientes, fraude no pagamento) e serve de porta de entrada para outros sistemas. Apagar o arquivo num commit seguinte **não resolve**, porque o valor continua no histórico e em todo clone ou fork. E o CI só roda depois do push: num repositório público, a chave já ficou exposta no branch antes de o Gitleaks reprovar. O CI impede que ela chegue ao `main`, mas um vazamento real ainda exige revogar e trocar a chave. Para barrar antes do push, o Gitleaks precisa rodar também como pre-commit (seção 9). O lugar certo da chave é um cofre (ou os GitHub Secrets), de onde ela é injetada no back-end durante o deploy. Nunca no código, e muito menos no front-end, onde qualquer visitante a leria.

Detalhe que reforça a necessidade do Gitleaks: a *push protection* nativa do GitHub está ligada neste repositório e **deixou esse push passar**, porque ela só reconhece padrões de provedores conhecidos (AWS, Stripe, tokens do GitHub…), e uma chave genérica não é um deles.

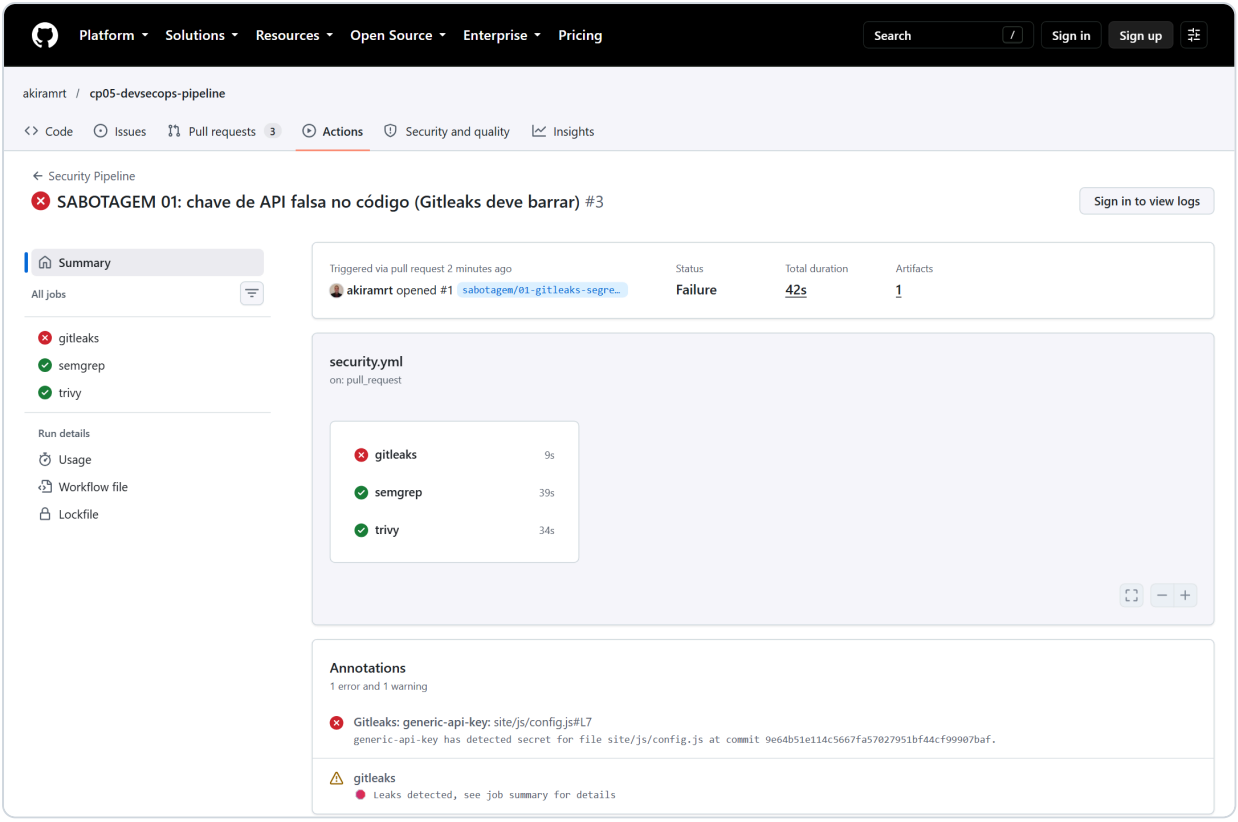


Figura 3: execução #3 de akiramrt/cp05-devsecops-pipeline (PR #1), iniciada em 22/10/2026 18:03:54 (Brasília); print às 18:06:37. gitleaks falhou, semgrep e trivy passaram; a anotação aponta `site/js/config.js#L7` · github.com/akiramrt/cp05-devsecops-pipeline/actions/runs/39854161173

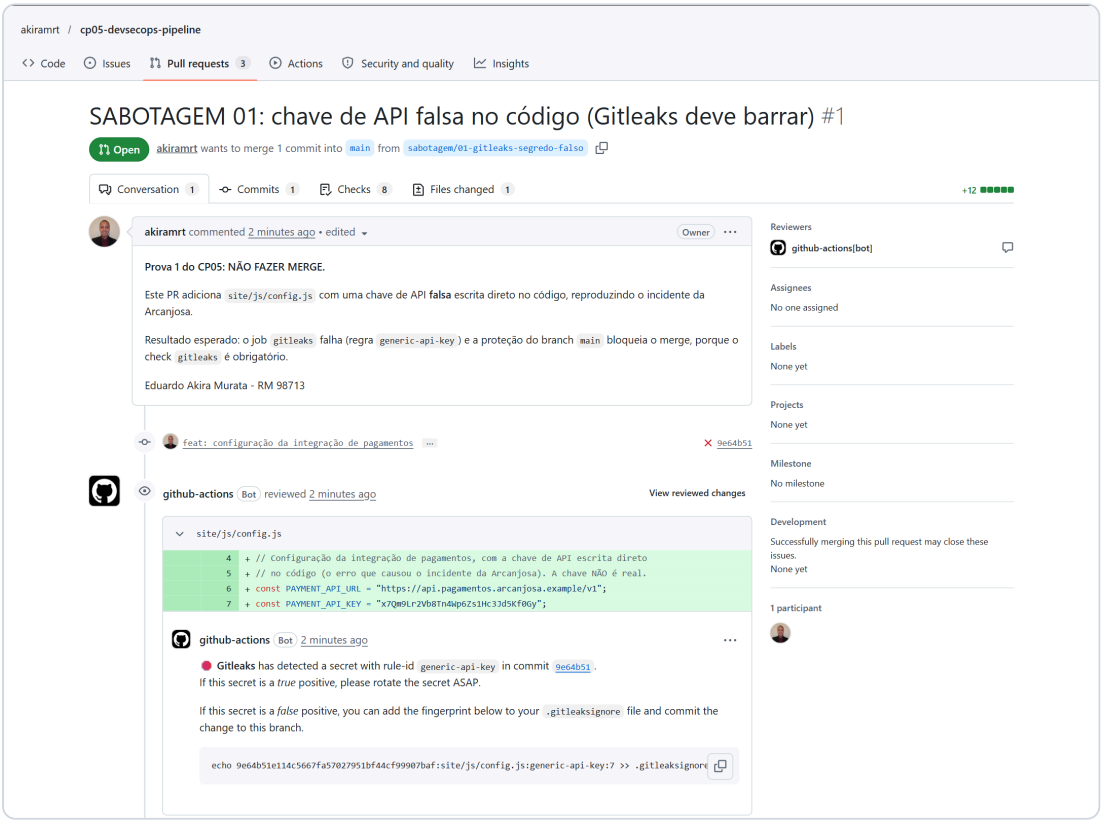


Figura 4: PR #1 de akiramrt (aberto às 18:03); print às 18:06:45. O bot do Gitleaks comenta direto na linha 7 do diff (regra generic-api-key, commit 9e64b51) · github.com/akiramrt/cp05-devsecops-pipeline/pull/1

Log do job `gitleaks` da execução #3, baixado pela API do GitHub (horários em UTC; Brasília = UTC−3).

```
21:04:00  gitleaks cmd: gitleaks detect --redact -v --exit-code=2 --report-format=sarif --report-path=results.sarif --log-level=debug --lo
21:04:00  Finding:     const PAYMENT_API_KEY = "REDACTED"
21:04:00  Secret:      REDACTED
21:04:00  RuleID:      generic-api-key
21:04:00  Entropy:     4.954196
21:04:00  File:        site/js/config.js
21:04:00  Line:        7
21:04:00  Commit:      9e64b51e114c5667fa57027951bf44cf99907baf
21:04:00  Author:      Eduardo Akira Murata
21:04:00  Date:        2026-10-22T21:03:29Z
21:04:00  Fingerprint: 9e64b51e114c5667fa57027951bf44cf99907baf:site/js/config.js:generic-api-key:7
21:04:00  9:04PM INF 1 commits scanned.
21:04:00  9:04PM WRN leaks found: 1
21:04:03  ##[warning]🛑 Leaks detected, see job summary for details
```

6. Barreira 2: Semgrep (código vulnerável) EVAL APONTADO

Sabotagem. No branch `sabotagem/02-semgrep-eval`, o commit `31edbc4` criou uma "calculadora rápida" (`site/js/calculadora.js`). A função `calcular(expressao)` devolve `eval(expressao)` e recebe tanto o texto digitado no campo quanto o que vier no `#hash` da URL.

O que o Semgrep detectou e por que isso é perigoso

O que detectou. O Semgrep faz análise estática (SAST): ele transforma o código em árvore sintática e procura *padrões estruturais*, não texto. Por isso `eval(x)`, `eval (x)` ou um `eval` quebrado em várias linhas casam do mesmo jeito, enquanto um comentário que só cita "eval" não casa. Duas regras apontaram a linha 8: a regra própria `cp05-no-eval` (severidade ERROR, CWE-95, em `.semgrep/no-eval.yml`) e a regra da comunidade `javascript.browser.security.eval-detected.eval-detected` (severidade WARNING), do pacote `p/default`. Com `--error`, qualquer achado, de qualquer severidade, faz o Semgrep sair com código 1: o job falha, as anotações aparecem no diff do PR e o SARIF gera alertas no Code Scanning. A regra própria existe para a barreira não depender só do pacote da comunidade, que pode mudar.

Por que é perigoso. `eval` executa uma string como código JavaScript, com todos os privilégios da página. Como a string vem do usuário, e também do fragmento da URL, basta enviar à vítima um link `…/#<código>` para rodar JavaScript no navegador dela: é XSS baseado em DOM, ou injeção de código (CWE-95; OWASP Top 10 A03:2021 – Injection, que na edição 2025 virou A05). Com isso o atacante lê dados da página, faz requisições em nome do usuário e altera o conteúdo para phishing. A correção é fazer o parsing explícito da entrada (aceitar só números e operadores e calcular com um parser), nunca `eval`. Como defesa em profundidade, a CSP do nginx não inclui `'unsafe-eval'`, então o navegador também bloquearia o `eval` em execução. Mas a CSP é a última linha de defesa; o pipeline barra antes, no PR.

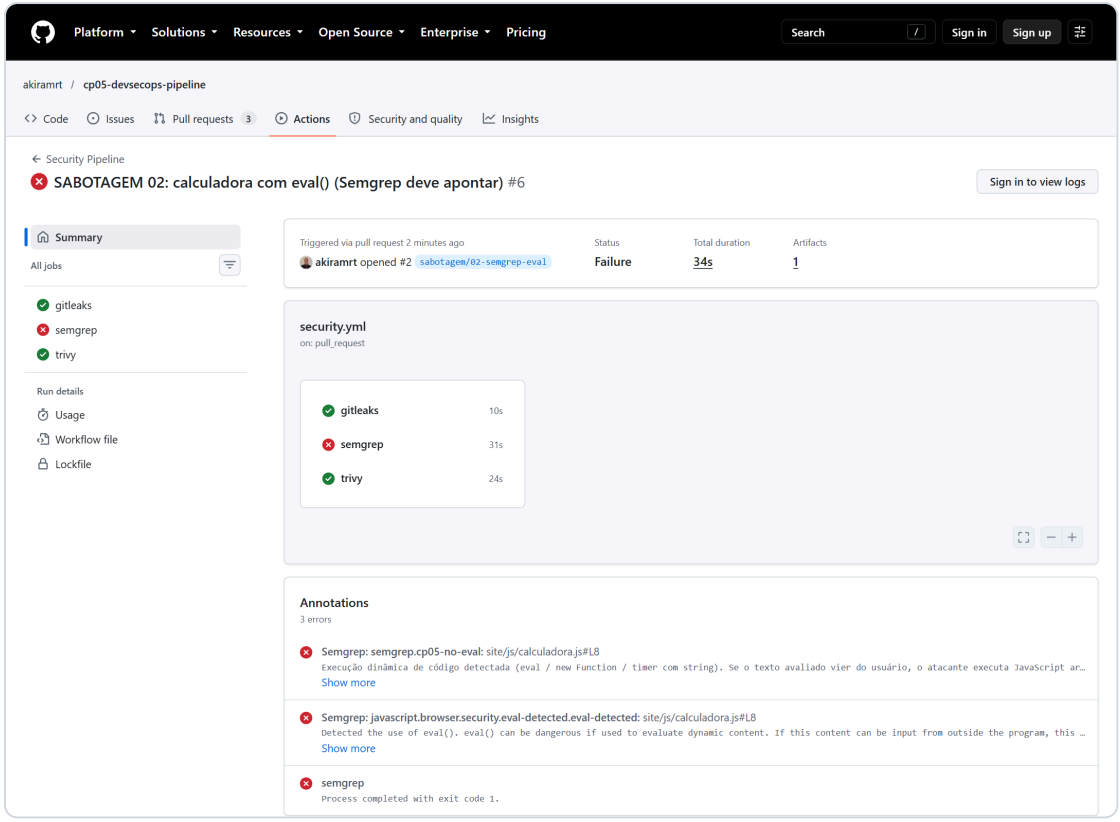


Figura 5: execução #6 de akiramrt/cp05-devsecops-pipeline (PR #2), iniciada em 22/10/2026 18:04:51 (Brasília); print às 18:07:42. semgrep falhou com 2 achados em `site/js/calculadora.js#L8` · github.com/akiramrt/cp05-devsecops-pipeline/actions/runs/39854294308

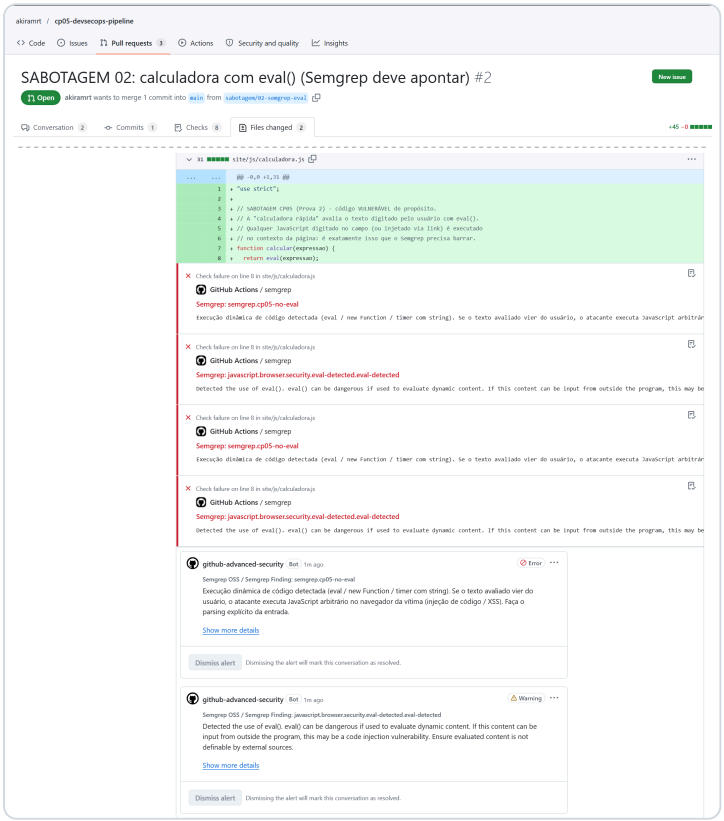


Figura 6: PR #2 de akiramrt, aba Files changed; print às 18:07:01. Os 2 achados aparecem na linha 8 do diff, cada um duas vezes (uma pela execução de push #4 e outra pela de PR #6), e o Code Scanning abre um alerta para cada regra · github.com/akiramrt/cp05-devsecops-pipeline/pull/2/files

Log do job `semgrep` da execução #6, baixado pela API do GitHub (horários em UTC; Brasília = UTC−3).

```
21:05:13
21:05:13  ┌─────────────────┐
21:05:13  │ 2 Code Findings │
21:05:13  └─────────────────┘
21:05:13
21:05:13      site/js/calculadora.js
21:05:13      ❯❱ javascript.browser.security.eval-detected.eval-detected
21:05:13            ❮❮ Blocking ❯❯
21:05:13            Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
21:05:13            content can be input from outside the program, this may be a code injection vulnerability. Ensure
21:05:13            evaluated content is not definable by external sources.
21:05:13            Details: https://sg.run/7ope
21:05:13
21:05:13              8┆ return eval(expressao);
21:05:13
21:05:13      ❯❯❱ semgrep.cp05-no-eval
21:05:13            ❮❮ Blocking ❯❯
21:05:13            Execução dinâmica de código detectada (eval / new Function / timer com string). Se o texto avaliado
21:05:13            vier do usuário, o atacante executa JavaScript arbitrário no navegador da vítima (injeção de código
21:05:13            / XSS). Faça o parsing explícito da entrada.
21:05:13
21:05:13              8┆ return eval(expressao);
21:05:13
21:05:13
21:05:13
21:05:13  ┌──────────────┐
21:05:13  │ Scan Summary │
21:05:13  └──────────────┘
21:05:13  ✅ Scan completed successfully.
21:05:13   • Findings: 2 (2 blocking)
21:05:13   • Rules run: 507
21:05:13   • Targets scanned: 14
21:05:13   • Parsed lines: ~100.0%
21:05:13   • Scan was limited to files tracked by git
21:05:13   • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
21:05:13  Ran 507 rules on 14 files: 2 findings.
21:05:13  ##[error]Process completed with exit code 1.
```

7. Barreira 3: Trivy (imagem com CVE crítico)

Imagem limpa `PASSOU`: em todo push e PR, o job constrói a imagem do site e o Trivy a escaneia. No `main` (execução #1, Figura 1 e log da seção 4), a base `nginx-unprivileged:1.31.8-alpine-slim` (Alpine 3.24.2, 21 pacotes) teve **0 CVEs CRITICAL com correção disponível**, que é o critério que reprova o job. Isso não quer dizer imagem zerada: CVEs de severidade menor vão para o SARIF da aba Security. Só depois disso o smoke test subiu o container e validou a configuração.

Bônus: imagem vulnerável `BARRADA`: no branch `sabotagem/03-trivy-imagem-vulneravel`, o commit `780b8ca` trocou a base por `nginx:1.16-alpine` (Alpine 3.10.4, sem suporte desde 2021).

O que o Trivy detectou e por que isso é perigoso

O que detectou. O Trivy abre as camadas da imagem, identifica o sistema operacional (Alpine 3.10.4, com o aviso "no longer supported"), lista os pacotes instalados a partir do banco de dados do `apk` e cruza as versões com bases de vulnerabilidades (Alpine secdb, NVD e outras). Ele encontrou 2 CVEs **CRITICAL com correção disponível**. O **CVE-2021-36159**, no `apk-tools` 2.10.4-r2, é uma leitura fora dos limites na libfetch, a biblioteca usada para baixar pacotes (corrigido na 2.10.7-r0). O **CVE-2020-15999**, no `freetype` 2.10.0-r0, é um estouro de buffer no heap (heap buffer overflow) no processamento de fontes, explorado como zero-day no Chrome em 2020 (corrigido na 2.10.0-r1). O Trivy saiu com código 1, o job falhou, o smoke test nem rodou e as anotações caíram na linha do `FROM`.

Por que é perigoso. O código do site pode estar impecável e a imagem ainda carregar bibliotecas de sistema vulneráveis, e é a imagem inteira que vai para produção. Uma falha crítica num pacote base costuma permitir execução de código ou queda do serviço a partir de uma entrada do atacante. Um sistema operacional sem suporte não recebe mais correções, então o problema só cresce.

Por que barrar só CRITICAL com correção. O pipeline bloqueia o que é *acionável*: existe versão corrigida, basta atualizar. As demais severidades com correção vão para o relatório SARIF na aba Security, para serem tratadas sem travar a entrega. Fixar a imagem base por digest torna o build reproduzível e faz da atualização uma mudança explícita e revisada.

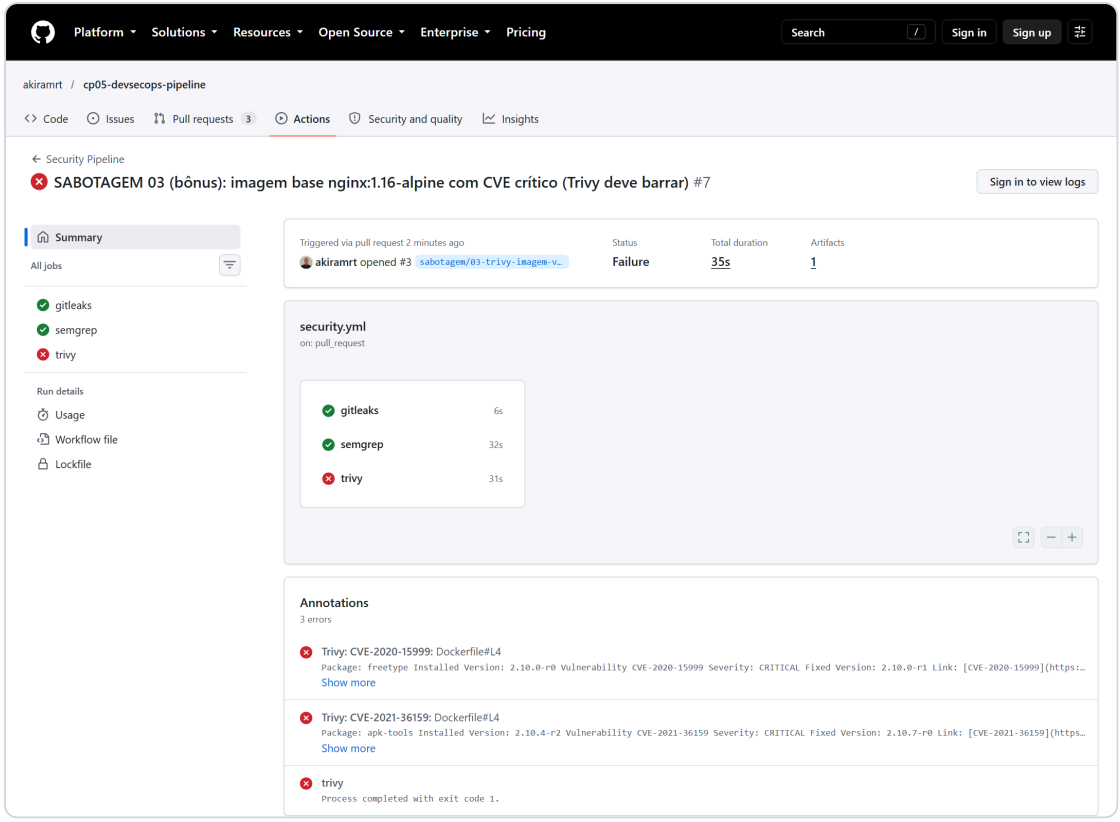


Figura 7: execução #7 de akiramrt/cp05-devsecops-pipeline (PR #3), iniciada em 22/10/2026 18:04:56 (Brasília); print às 18:07:48. trivy falhou com 2 CVEs CRITICAL anotados na linha do `FROM` (Dockerfile#L4) · github.com/akiramrt/cp05-devsecops-pipeline/actions/runs/39854305569

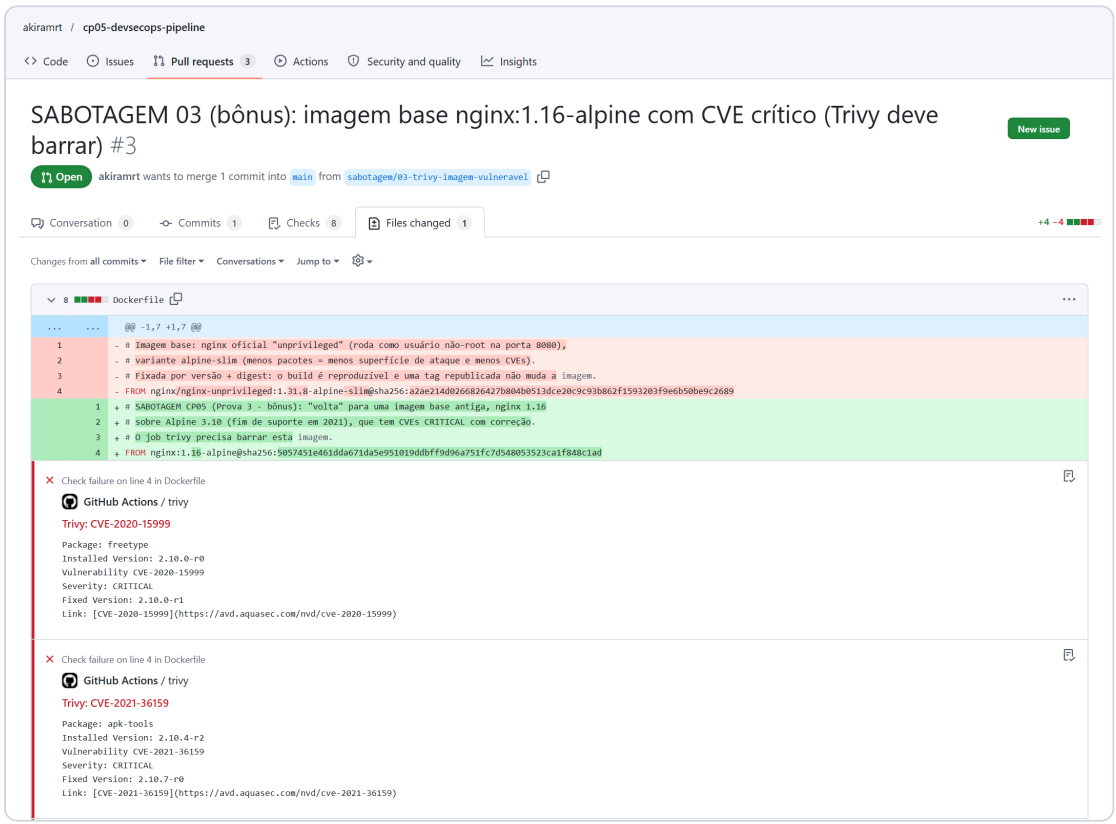


Figura 8: PR #3 de akiramrt, aba Files changed; print às 18:07:56. As falhas do Trivy aparecem na linha 4 do Dockerfile, a linha do FROM (cada CVE duas vezes: execuções de push e de PR) · github.com/akiramrt/cp05-devsecops-pipeline/pull/3/files

Log do job `trivy` da execução #7, baixado pela API do GitHub (horários em UTC; Brasília = UTC−3). A tag da imagem é `e1627ad`: num `pull_request`, o `github.sha` é o commit de merge de teste que o GitHub cria entre o PR e o `main`. O commit do branch é `780b8ca`.

```
21:05:18  2026-10-22T21:05:18Z  INFO    Detected OS     family="alpine" version="3.10.4"
21:05:18  2026-10-22T21:05:18Z  WARN    This OS version is no longer supported by the distribution      family="alpine" version="3.10.4"
21:05:18  arcanjosa-site:e1627ad48c09be08bc09488666bc87f9fc758eba (alpine 3.10.4)
21:05:18  ═══════════════════════════════════════════════════════════════════════
21:05:18  Total: 2 (CRITICAL: 2)
21:05:18
21:05:18  ┌─────────┬────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────┐
21:05:18  │ Library │ Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                Title                 │
21:05:18  ├─────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────┤
21:05:18  │ apk-tools │ CVE-2021-36159 │ CRITICAL │ fixed │ 2.10.4-r2 │ 2.10.7-r0 │ libfetch: an out of boundary read while l
21:05:18  │         │                │          │        │                   │               │ to parse ...
21:05:18  │         │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2021-3615
21:05:18  ├─────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────┤
21:05:18  │ freetype │ CVE-2020-15999 │          │        │ 2.10.0-r0 │ 2.10.0-r1 │ freetype: Heap-based buffer overflow due
21:05:18  │         │                │          │        │                   │               │ truncation in Load_SBit_Png
21:05:18  │         │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2020-1599
21:05:18  └─────────┴────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────┘
21:05:18
21:05:18  ##[error]Process completed with exit code 1.
```

8. O merge fica bloqueado

Um pipeline que só avisa vira relatório que ninguém lê. Por isso o branch `main` tem uma regra de proteção que exige os três checks aprovados, vindos do app GitHub Actions (`app_id 15368`; assim um check com o mesmo nome criado por outra integração não consegue satisfazer a regra). A regra vale também para administradores (`enforce_admins`), então nem eu, dono do repositório, consigo fazer merge com um check vermelho. Também estão bloqueados o force-push e a exclusão do branch. Configuração lida da API do GitHub no momento em que este PDF foi gerado:

```
GET /repos/akiramrt/cp05-devsecops-pipeline/branches/main/protection
required_status_checks.checks = [
  { "context": "gitleaks", "app_id": 15368 },
  { "context": "semgrep",  "app_id": 15368 },
  { "context": "trivy",    "app_id": 15368 } ]
enforce_admins.enabled = true · allow_force_pushes = false · allow_deletions = false
```

Estado dos três pull requests de sabotagem, também pela API:

```
PR #1 sabotagem/01-gitleaks-segredo-falso    state=open  merged=false  mergeable_state=blocked
PR #2 sabotagem/02-semgrep-eval              state=open  merged=false  mergeable_state=blocked
PR #3 sabotagem/03-trivy-imagem-vulneravel   state=open  merged=false  mergeable_state=blocked
```

Os PRs ficam abertos, sem merge, como evidência. Nenhuma das sabotagens chegou ao `main`. A própria entrega deste PDF entrou no `main` pelo caminho normal: um PR de documentação que só foi integrado depois que os três checks passaram.

9. Conclusão e próximos passos

O pipeline cumpre o que a diretoria pediu: barra segredos, código vulnerável e imagem com CVE crítico, e **prova** isso com sabotagens documentadas. Em cada uma falhou exatamente a barreira responsável, com arquivo, linha e regra apontados, e o merge ficou bloqueado. O pipeline também foi construído com cuidado de supply chain (actions por SHA, imagens por digest, ferramentas em versões sem CVE conhecido, menor privilégio) e falha fechado quando não consegue verificar.

Limitações conhecidas e melhorias possíveis:

Mais cedo no ciclo	O CI roda depois do push. Gitleaks como hook de pre-commit barra o segredo antes de ele sair da máquina. Push protection + pre-commit + CI formam camadas.
Pacote `p/default` baixado na hora	Regras novas da comunidade podem mudar o resultado sem mudança no repositório. Dá para versionar as regras no próprio repo, se for preciso reprodutibilidade total.
CVE sem correção	Com `ignore-unfixed`, um CVE CRITICAL sem versão corrigida não barra o job nem aparece no SARIF. Vale acompanhar esses casos à parte, por exemplo com um scan semanal sem esse filtro.
Binário do Gitleaks	A action baixa o Gitleaks sem conferir checksum. A alternativa é instalar o CLI 8.30.1 verificando o SHA-256 publicado.
Atualizações	Dependabot para os digests da imagem base e os SHAs das actions, e a opção "Require actions to be pinned to a full-length commit SHA" nas configurações do repositório.

Anexo A: `.github/workflows/security.yml`

```yaml
name: Security Pipeline

# Roda em todo push (qualquer branch) e em todo pull request, como pede o CP05.
on: [push, pull_request]

# Menor privilégio por padrão: cada job pede só a permissão extra de que precisa.
permissions:
  contents: read

# Um push novo no mesmo PR cancela a execução anterior desse PR.
# Cada push tem o próprio grupo (run_id), então execuções de push nunca se cancelam.
concurrency:
  group: security-${{ github.event_name == 'pull_request' && format('pr-{0}',
github.event.pull_request.number) || format('push-{0}', github.run_id) }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

# Todas as actions de terceiros estão fixadas por SHA completo do commit (não por tag).
# Motivo: em março/2026 as tags do aquasecurity/trivy-action foram reescritas por um
# atacante (GHSA-69fq-xp46-6x23). Um SHA não pode ser trocado por baixo do pipeline.

jobs:
  # ─────────────────────────────────────────────────────────────────────────
  # Barreira 1: SEGREDOS. O Gitleaks varre os commits enviados no push / no PR
  # procurando chaves de API, tokens, chaves privadas e senhas de alta entropia.
  # ─────────────────────────────────────────────────────────────────────────
  gitleaks:
    name: gitleaks
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    permissions:
      contents: read
      pull-requests: write # o gitleaks comenta no PR, na linha do segredo
    steps:
      - name: Checkout (histórico completo, o gitleaks lê os commits)
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          fetch-depth: 0
          persist-credentials: false

      # A gitleaks-action monta o intervalo "<primeiro>^..<último>" dos commits do push.
      # Se o primeiro commit for a raiz do repositório, "<raiz>^" não existe: o gitleaks
      # varre 0 commits e o job passaria VERDE sem verificar nada. Aqui ele falha fechado.
      - name: Guarda - o gitleaks não pode varrer 0 commits
        if: github.event_name == 'push'
        run: |
          read -r total first <<< "$(python3 -c 'import json, os; c =
json.load(open(os.environ["GITHUB_EVENT_PATH"])).get("commits") or []; print(len(c), c[0]["id"] if c else "-
")')"
          echo "Commits neste push: $total (primeiro: $first)"
          if [ "$total" -gt 1 ] && ! git rev-parse --verify --quiet "${first}^" > /dev/null; then
            echo "::error title=gitleaks::O push começa no commit raiz ${first}; o gitleaks varreria 0
commits. Envie os commits em partes."
            exit 1
          fi

      - name: Gitleaks - procura segredos nos commits
        uses: gitleaks/gitleaks-action@e0c47f4f8be36e29cdc102c57e68cb5cbf0e8d1e # v3.0.0
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          GITLEAKS_VERSION: "8.30.1" # a versão padrão da action (8.24.3) tem a CVE-2026-63728
          GITLEAKS_ENABLE_SUMMARY: "true"
          GITLEAKS_ENABLE_COMMENTS: "true"
          GITLEAKS_ENABLE_UPLOAD_ARTIFACT: "true"

      - name: Anotar segredos encontrados
        if: failure()
        run: python3 .github/scripts/sarif_report.py results.sarif --tool Gitleaks
```

```yaml
  # ──────────────────────────────────────────────────────────────────────
  # Barreira 2: CÓDIGO VULNERÁVEL. O Semgrep faz análise estática (SAST) com as
  # regras da comunidade (p/default) + a regra própria .semgrep/no-eval.yml.
  # --error faz o job falhar se houver qualquer achado.
  # ──────────────────────────────────────────────────────────────────────
  semgrep:
    name: semgrep
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    permissions:
      contents: read
      security-events: write # envio do SARIF para a aba Security > Code scanning
    container:
      image: semgrep/semgrep:1.181.1@sha256:3db5c7af55eb1f512eac0abf0ab69a97b52c74b5e9a27314c221aee1a78d7ab2
    steps:
      - name: Checkout
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false

      # O container roda como root e o workspace pertence ao usuário do runner
      - name: Git safe.directory
        run: git config --global --add safe.directory "$GITHUB_WORKSPACE"

      - name: Semgrep - análise estática (SAST)
        run: |
          semgrep scan \
            --config p/default \
            --config .semgrep/ \
            --metrics=off \
            --error \
            --sarif-output=semgrep.sarif

      - name: Anotar achados do Semgrep
        if: ${{ !cancelled() }}
        run: python3 .github/scripts/sarif_report.py semgrep.sarif --tool Semgrep

      - name: Enviar SARIF para o Code Scanning
        if: ${{ !cancelled() && (github.event_name ≠ 'pull_request' ||
github.event.pull_request.head.repo.full_name == github.repository) }}
        uses: github/codeql-action/upload-sarif@9b3254085494b55a0f7c4c2b3d2d52cc712ce370 # v4.39.1
        with:
          sarif_file: semgrep.sarif
          category: semgrep

  # ──────────────────────────────────────────────────────────────────────
  # Barreira 3: IMAGEM COM CVE CRÍTICO. Constrói a imagem do Dockerfile e o Trivy
  # escaneia SO e bibliotecas. Falha só com CVE CRITICAL que já tem correção.
  # ──────────────────────────────────────────────────────────────────────
  trivy:
    name: trivy
    runs-on: ubuntu-24.04
    timeout-minutes: 20
    permissions:
      contents: read
      security-events: write
    env:
      IMAGE: arcanjosa-site:${{ github.sha }}
    steps:
      - name: Checkout
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false

      - name: Docker build da imagem do site
        id: build
        run: docker build --pull --tag "$IMAGE" .

      - name: Trivy - barra CVE CRITICAL com correção disponível
        uses: aquasecurity/trivy-action@ed142fd0673e97e23eac54620cfb913e5ce36c25 # v0.36.0
```

```
        env:
          TRIVY_IMAGE_SRC: docker
        with:
          version: v0.76.1
          scan-type: image
          image-ref: ${{ env.IMAGE }}
          scanners: vuln
          vuln-type: os,library
          severity: CRITICAL
          ignore-unfixed: true
          format: table
          exit-code: "1"

      # Só roda se a imagem passou no Trivy: prova que o container sobe e serve o site.
      - name: Smoke test do container (nginx -t, /healthz, headers)
        run: |
          docker run --rm "$IMAGE" nginx -t
          docker run -d --name site --read-only --tmpfs /tmp --cap-drop ALL \
            --security-opt no-new-privileges -p 8080:8080 "$IMAGE"
          for i in $(seq 1 20); do curl -fsS http://127.0.0.1:8080/healthz && break; sleep 1; done
          curl -fsS http://127.0.0.1:8080/healthz
          curl -fsSI http://127.0.0.1:8080/ | tee headers.txt
          grep -qi '^content-security-policy:' headers.txt
          grep -qi '^x-frame-options: deny' headers.txt
          test "$(curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:8080/.git/config)" = 403
          docker exec site id -u | grep -qx 101
          docker rm -f site

      - name: Trivy - relatório SARIF (todas as severidades com correção)
        if: ${{ !cancelled() && steps.build.outcome == 'success' }}
        uses: aquasecurity/trivy-action@ed142fd0673e97e23eac54620cfb913e5ce36c25 # v0.36.0
        env:
          TRIVY_IMAGE_SRC: docker
          TRIVY_SKIP_DB_UPDATE: "true"
          TRIVY_SKIP_JAVA_DB_UPDATE: "true"
        with:
          skip-setup-trivy: true
          cache: "false" # reaproveita o banco que o passo anterior acabou de baixar
          scan-type: image
          image-ref: ${{ env.IMAGE }}
          scanners: vuln
          ignore-unfixed: true
          format: sarif
          output: trivy-results.sarif
          exit-code: "0"

      - name: Anotar CVEs críticos
        if: ${{ !cancelled() && steps.build.outcome == 'success' }}
        run: |
          from_line="$(grep -n -m1 '^FROM' Dockerfile | cut -d: -f1)"
          python3 .github/scripts/sarif_report.py trivy-results.sarif --tool Trivy --only-tag CRITICAL --
anchor "Dockerfile:${from_line:-1}"

      - name: Enviar SARIF para o Code Scanning
        if: ${{ !cancelled() && steps.build.outcome == 'success' && (github.event_name ≠ 'pull_request' ||
github.event.pull_request.head.repo.full_name == github.repository) }}
        uses: github/codeql-action/upload-sarif@9b3254085494b55a0f7c4c2b3d2d52cc712ce370 # v4.39.1
        with:
          sarif_file: trivy-results.sarif
          category: trivy-image
```

Anexo B: `.semgrep/no-eval.yml`

```
rules:
  - id: cp05-no-eval
    languages: [javascript, typescript]
    severity: ERROR
    message: >-
      Execução dinâmica de código detectada (eval / new Function / timer com string).
```

```
      Se o texto avaliado vier do usuário, o atacante executa JavaScript arbitrário
      no navegador da vítima (injeção de código / XSS). Faça o parsing explícito da entrada.
    pattern-either:
      - pattern: eval(...)
      - pattern: window.eval(...)
      - pattern: globalThis.eval(...)
      - pattern: new Function(...)
      - pattern: Function(...)
      - pattern: setTimeout("...", ...)
      - pattern: setInterval("...", ...)
    metadata:
      category: security
      cwe:
        - "CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')"
      owasp:
        - "A03:2021 - Injection"
      references:
        - https://developer.mozilla.org/en-
US/docs/Web/JavaScript/Reference/Global_Objects/eval#never_use_direct_eval!
```

Anexo C: links

Repositório	https://github.com/akiramrt/cp05-devsecops-pipeline
Execuções (Actions)	https://github.com/akiramrt/cp05-devsecops-pipeline/actions
Pull requests de sabotagem	#1 Gitleaks · #2 Semgrep · #3 Trivy
Prints em alta resolução	https://github.com/akiramrt/cp05-devsecops-pipeline/tree/main/docs/evidencias